Lesson 4: Direct Hit

Unity

VIDEO GAME CREATION

This week: detecting when things touch. Your job is to add one enemy to the screen and make your bullet destroy it on contact - the first piece of your capstone's actual combat loop.

Open your own [YourName]Game project from Lesson 3 - don't start a new one. Everything this week builds on the player and bullet you already have.

Tip: this lesson's Assets folder has a ready-to-use placeholder enemy image (enemy_invader_red.png) and a new sound effect (sfx_hit.wav) - you don't need to find or make your own art today.

Setup

1. Open your [YourName]Game project from Lesson 3.
2. On the Bullet prefab, add a Box Collider 2D component and tick Is Trigger. Add a Rigidbody2D too, Body Type set to Kinematic - Unity's 2D physics needs at least one Rigidbody2D between two colliding objects for trigger events to fire, and Bullet is the one already moving each frame.
3. Create a new empty GameObject, add a Sprite Renderer and assign enemy_invader_red.png (import it from this lesson's Assets folder) as its Sprite. Add a Box Collider 2D and tick Is Trigger.
4. In the Inspector's Tag dropdown, click Add Tag, create a new tag named Enemy, then assign it to this GameObject.
5. Drag this GameObject into the Project window to make it a Prefab named Enemy, then position an instance of it in the scene near the top of the play area.

Level 1 - Minimal (MVP)

A single enemy sits on screen. When your bullet touches it, both the bullet and the enemy disappear. This is a complete finish for today - reaching this is enough to move on to next week.

6. In Bullet.cs, add an OnTriggerEnter2D(Collider2D other) method that checks other.CompareTag("Enemy") and, if true, destroys both the other object and itself (code below).
7. Press Play and shoot the Enemy to test - both the bullet and the Enemy should disappear on contact.

```
// Bullet.cs - add this method:
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
```

```
        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}
```

Alternate Path: Visual Scripting

The same trigger-detection pattern works in a Script Graph instead of C# - Unity's Visual Scripting graphs compile to the same underlying calls.

8. On the Bullet prefab, add an On Trigger Enter 2D event node, feeding a Branch node that checks the other Collider's Compare Tag against "Enemy".
9. On True, wire a Destroy node targeting the other Collider's Game Object, and a second Destroy node targeting this Game Object.
10. Set up the Bullet's Rigidbody2D and both colliders' Is Trigger exactly as in the C# path - the physics requirements are the same no matter which scripting style triggers the event.

Level 3's ideas are all reachable with more nodes: an Audio Source "Play Clip At Point" node for sound, a "Set Member" node on the Sprite Renderer's color for the Enemy's own visual idea - same ideas as the C# path, just wired instead of typed.

Level 2 - Complete (Alpha/Beta)

The enemy is destroyed cleanly and exactly once, even if you fire several bullets in a row - no errors, no lingering references, and a bullet that misses still behaves exactly like it did last week.

- Fire several bullets in a row, including some that miss - each hit destroys exactly one bullet and the Enemy, and any bullet that misses keeps flying and destroys itself off-screen exactly like it did in Lesson 3.
- Destroy() doesn't remove a GameObject instantly - it happens at the end of the current frame - so if two bullets somehow trigger on the exact same frame, both destroy calls on the Enemy are harmless; Unity simply ignores the second one, and no error is thrown.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: import sfx_hit.wav, add a public AudioClip hitSound field to Bullet.cs, and call AudioSource.PlayClipAtPoint(hitSound, transform.position) inside OnTriggerEnter2D before either object is destroyed - this static method spawns its own temporary audio source that survives independently of the Enemy or Bullet being destroyed, so the sound always finishes playing.
- Visual: give the Enemy prefab its own SpriteRenderer.color (distinct from the bullet's yellow) in a small new Enemy.cs script's Start(), so it clearly reads as a threat rather than another shot.
- Structure: OnTriggerEnter2D in Bullet.cs should only ever detect a hit and clean up (destroy the enemy, destroy itself, play the sound) - anything about how the Enemy looks on its own belongs in its own Enemy.cs, not bolted onto Bullet.cs.

If Something's Wrong

- Nothing happens on contact: confirm both colliders have Is Trigger checked, and that the Bullet prefab actually has a Rigidbody2D - two Collider2Ds with no Rigidbody2D between them never generate trigger events in Unity's 2D physics.

- Console error about a missing tag: confirm the Enemy tag was actually created and assigned (Inspector's Tag dropdown, top-left) on the Enemy prefab itself, not just typed as a string somewhere.
- Bullet destroys itself on everything, not just the Enemy: double-check the CompareTag("Enemy") check - without it, OnTriggerEnter2D fires for anything the bullet's collider touches.

Want More?

Student Resource List > Unity section, or learn.unity.com